

Computational Statistics for Data Analysis

✓ "Statistics is the discipline of using data samples to support claims about populations."

In this notebook, we will get familiar with Descriptive statistics that comprise concepts, terms, measures and tools that help to describe, show and summarize the data in a meaningful way. When analysing data, such as the price of flats renting per year, it is possible to use both descriptive and inferential statistics in order to analyse the results and draw some conclusions. We will discuss basic concepts, terms and procedures, like mean, median, variance, correlation, etc., to explore, describe and summarize the given set of data.

Statistics is based on 2 main concepts:

- A **population** is a collection of objects, items ("units") about which information is sought.
- A **sample** is a part of the population that is observed.

1 Descriptive Statistics. * 1.1 Getting data * 1.2 Data preparation * 1.3 Improving data as a pandas DataFrame * 1.4 Data cleaning

"Are the men more likely to become high income professionals i.e. to receive income bigger than 50K?"

Some people believe it is true, but **without data analysis** to support it, this claim is a case of **anecdotal evidence**:

- There are a **small number of samples** (personal experience, friends, etc.).
- There is a **selection bias**: most *believers* are interested in this claim because their first babies were late.
- There is a **confirmation bias**: believers might be more likely to contribute data that confirm it.
- Sources are **innaccurate**: personal stories are subject to memory deformations.

✓ 1.1 Data

Lab2_DataFile contains approximately 32.000 observations about different financial parameters of US population.

✓ 1.2 Data preparation

One of the reasons we are using a general-purpose language such as Python rather than a stats language like R is that for many projects the *hard* part is preparing the data, not doing the analysis.

The most common steps are:

1. **Getting the data.** Data can be directly read from a file or it might be necessary to scrap the web.
2. **Parsing the data.** Of course, this depends on what format it is in: plain text, fixed columns, CSV, XML, HTML, etc.
3. **Cleaning the data.** Survey responses and other data files are almost always incomplete. Sometimes there are multiple codes for things like, *not asked*, *did not know*, and *declined to answer*. And there are almost always errors. A simple strategy is to remove or ignore incomplete records.
4. **Building data structures.** Once you read the data, you usually want to store it in a data structure that lends itself to the analysis you want to do.

If the data fits into memory, building a data structure is usually the way to go. If not, you could build a database, which is an out-of-memory data structure. Most databases provide a mapping from keys to values, so they are like dictionaries.

Let us read the dataset:

```
file = open('Lab2_DataFile', 'r')
```

```
def chr_int(a):
    if a.isdigit():
        return int(a)
    else:
        return 0

data=[]
for line in file:
    data1=line.split(' ')
    if len(data1)==15:
        data.append([chr_int(data1[0]),data1[1],chr_int(data1[2]),data1[3],chr_int(data1[4]),data1[5],data1[6],\
                    data1[7],data1[8],data1[9],chr_int(data1[10]),chr_int(data1[11]),chr_int(data1[12]),data1[13],\
                    data1[14]])
```

```
print(data[1:2])
```

[Show hidden output](#)

1.3 Importing data as a pandas DataFrame

```
%matplotlib inline
import pandas as pd

df = pd.DataFrame(data) # Two-dimensional size-mutable, potentially heterogeneous tabular data structure with 1
df.columns = ['age', 'type_employer', 'fnlwgt', 'education',
              "education_num", "marital", "occupation", "relationship", "race", "gender",
              "capital_gain", "capital_loss", "hr_per_week", "country", "income"]
df.head()
```

[Show hidden output](#)

```
df.tail()
```

[Show hidden output](#)

```
df.shape
```

[Show hidden output](#)

Let's count the number of items per country:

df.Column_Name.value_counts() *-Counts unique occurrences *-Sorts in descending order *-Excludes missing values

```
counts = df.groupby('country').size()
print(counts)
```

```
# also:
df.country.value_counts()
```

[Show hidden output](#)

```
counts = df.groupby('age').size() # grouping by age
print(counts)
```

[Show hidden output](#)

```
ml = df[(df.gender == 'Male')] # grouping by gender
ml.shape
```

[Show hidden output](#)

```
ml50 = df[(df.gender == 'Male') & (df.income == '>50K\n')]
ml50.shape
```

[Show hidden output](#)

Let's separate male from female according to the income.

```
fm = df[(df.gender == 'Female')]
fm.shape
```

[Show hidden output](#)

```
fm50 = df[(df.gender == 'Female') & (df.income == '>50K\n')]
fm50.shape
```

[Show hidden output](#)

```
df50 = df[(df.income == '>50K\n')]

print('The rate of people with high income is: ', int(len(df50)/float(len(df))*100), '%.')
print('The rate of men with high income is: ', int(len(ml50)/float(len(ml))*100), '%.')
print('The rate of women with high income is: ', int(len(fm50)/float(len(fm))*100), '%.')
```

1.4 Data Cleaning

The most common steps are:

- **Sample the data.** If the amount of raw data is huge, processing all of them may require an extensive amount of processing power which may not be practical. In this case, it is quite common to sample the input data to reduce the size of data that need to be processed.
- **Impute missing data.** It is quite common that some of the input records are incomplete in the sense that certain fields are missing or have input error. In a typical tabular data format, we need to validate each record contains the same number of fields and each field contains the data type we expect. In case the record has some fields missing, we have the following choices:

* (a) Discard the whole record if it is incomplete; * (b) Infer the missing value based on the data from other records. A common approach is to fill the missing data with the average, or the median.

- **Normalize numeric value.** Normalize data is about transforming numeric data into a uniform range.
- **Reduce dimensionality.** High dimensionality can be a problem for some machine learning methods. There are two ways to reduce the number of input features. One is about *removing irrelevant* input variables, another one is about *removing redundant* input variables.
- **Add derived features.** In some cases, we may need to compute additional attributes from existing attributes (f.e. converting a geo-location to a zip code, or converting the age to an age group).
- **Discretize numeric value into categories.** Discretize data is about cutting a continuous value into ranges and assigning the numeric with the corresponding bucket of the range it falls on. For numeric attribute, a common way to generalize it is to discretize it into ranges, which can be either constant width (variable height/frequency) or variable width (constant height).
- **Binarize categorical attributes.** Certain machine learning models only take binary input (or numeric input). In this case, we need to convert categorical attribute into multiple binary attributes, while each binary attribute corresponds to a particular value of the category.
- **Select, combine, aggregate data.** Designing the form of training data is the most important part of the whole predictive modeling exercise because the accuracy largely depends on whether the input features are structured in an appropriate form that provide strong signals to the learning algorithm. Rather than using the raw data as it is, it is quite common that multiple pieces of raw data need to be combined together, or aggregating multiple raw data records along some dimensions.

2 Exploratory Data Analysis.

2.1 Summarizing the data:

2.1.1 Sample Mean

If you have a sample of n values, x_i , the **sample mean** is the sum of the values divided by the number of values:

$$\mu = \frac{1}{n} \sum_i x_i$$

The **mean** is the most basic and important summary statistic. It describes the central tendency of a sample.

There is a small difference!

```
print('The average age of men is: ', ml['age'].mean(), '.')
print('The average age of women is: ', fm['age'].mean(), '.')
```

[Show hidden output](#)

This difference in sample means can be considered a first evidence of our hypothesis!

****Comment:** *Later, we will work with both concepts: the population mean and the sample mean. Do not confuse them! Remember, the first one is the mean of samples taken from the population and the second one is the mean of the whole population.*

```
print('The average age of high-income men is: ', ml50['age'].mean(), '.')
print('The average age of high-income women is: ', fm50['age'].mean(), '.')
```

[Show hidden output](#)

2.1.2 Sample Variance

Usually, mean is not a sufficient descriptor of the data, we can do a little better with two numbers: mean and **variance**:

$$\sigma^2 = \frac{1}{n} \sum_i (x_i - \mu)^2$$

Variance σ^2 describes the *spread* of data. The term $(x_i - \mu)$ is called the *deviation from the mean*, so variance is the mean squared deviation.

The square root of variance, σ , is called the **standard deviation**. We define standard deviation because variance is hard to interpret (in the case the units are grams, the variance is in grams squared). Let's get the basic statistics for our example data:

```
ml_mu = ml['age'].mean()
fm_mu = fm['age'].mean()
ml_var = ml['age'].var()
fm_var = fm['age'].var()
ml_std = ml['age'].std()
fm_std = fm['age'].std()
print('Statistics of age for men: mu:', ml_mu, 'var:', ml_var, 'std:', ml_std)
print('Statistics of age for women: mu:', fm_mu, 'var:', fm_var, 'std:', fm_std)
```

[Show hidden output](#)

```
ml_mu_hr = ml['hr_per_week'].mean()
fm_mu_hr = fm['hr_per_week'].mean()
ml_var_hr = ml['hr_per_week'].var()
fm_var_hr = fm['hr_per_week'].var()
ml_std_hr = ml['hr_per_week'].std()
fm_std_hr = fm['hr_per_week'].std()
print('Statistics of hours per week for men: mu:', ml_mu_hr, 'var:', ml_var_hr, 'std:', ml_std_hr)
print('Statistics of hours per week for women: mu:', fm_mu_hr, 'var:', fm_var_hr, 'std:', fm_std_hr)
```

[Show hidden output](#)

2.1.3 Sample Median

The statistical median is an order statistic that gives the *middle* value of a sample. It is a value more robust to outliers.

```
ml_median= ml['age'].median()
fm_median= fm['age'].median()
print("Median age per men and women: ", ml_median, fm_median)

ml_median_age= ml50['age'].median()
fm_median_age= fm50['age'].median()
print("Median age per men and women with high-income: ", ml_median_age, fm_median_age)
```

[Show hidden output](#)

```
ml_median_hr= ml['hr_per_week'].median()
fm_median_hr= fm['hr_per_week'].median()
print("Median hours per week per men and women: ", ml_median_hr, fm_median_hr)
```

[Show hidden output](#)

2.1.4 Summarizing the data: Quantiles & Percentiles

Order the sample $\{x_i\}$, then find x_p so that it divides the data into two parts where:

- a fraction p of the data values are less than or equal to x_p and
- the remaining fraction $(1 - p)$ are greater than x_p .

That value x_p is the p th-quantile, or $100 \times p$ th percentile.

5-number summary: x_{min} , Q_1 , Q_2 , Q_3 , x_{max} , where Q_1 is the 25×pth percentile, Q_2 is the 50×pth percentile and Q_3 is the 75×pth percentile.

2.2 Histogram

The most common representation of a distribution is a **histogram**, which is a graph that shows the frequency of each value. Let us visualize the histogram for the age of the male and female populations in our example:

```
import matplotlib.pyplot as plt
ml_age=ml['age']
```

```
ml_age.hist(density=False, histtype='stepfilled', bins=5)

plt.xlabel('Age', fontsize=15)
plt.ylabel('Male samples', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
fm_age=fm['age']
fm_age.hist(density=False, histtype='stepfilled', bins=10)
plt.xlabel('Age', fontsize=15)
plt.ylabel('Female samples', fontsize=15)
plt.show()
```

[Show hidden output](#)

Let's compare both populations:

```
import seaborn as sns
fm_age.hist(density=False, histtype='stepfilled', alpha=.5, bins=10) # default number of bins = 10
ml_age.hist(density=False, histtype='stepfilled', alpha=.5, color=sns.desaturate("indianred", .75), bins=10)
plt.xlabel('Age', fontsize=15)
plt.ylabel('Samples', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
fm_age.hist(density=True, histtype='stepfilled', alpha=.5, bins=20) # default number of bins = 10
ml_age.hist(density=True, histtype='stepfilled', alpha=.5, color=sns.desaturate("indianred", .75), bins=10)
plt.xlabel('Age', fontsize=15)
plt.ylabel('PMF', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
import scipy.stats as stats
```

2.3 Data Distributions

Summarizing can be dangerous: very different data can be described by the same statistics. It must be validated by inspecting the data.

We can look at the **data distribution**, which describes how often (frequency) each value appears.

We can normalize the frequencies of the histogram by dividing/normalizing by n , the number of samples. The normalized histogram is called **Probability Mass Function (PMF)**.

Let's visualize and compare the MPF of male and female age in our example:

```
ml_age.hist(density=True, histtype='stepfilled', bins=20)
plt.xlabel('Age', fontsize=15)
plt.ylabel('Probability', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
fm_age.hist(density=True, histtype='stepfilled', bins=20)
plt.xlabel('Age', fontsize=15)
plt.ylabel('Probability', fontsize=15)
plt.show()
```

[Show hidden output](#)

The **cumulative distribution function (CDF)**, or just distribution function, describes the probability that a real-valued random variable X with a given probability distribution will be found to have a value less than or equal to x .

cumulative=True This computes the cumulative histogram where each bin's value represents the count (or density) of all data points in that bin plus all data points in bins with smaller values. When combined with `density=True`, the y-axis shows the probability of a data point being less than or equal to the corresponding x-value (the CDF). The final y-value will be 1.0.

For our example, the CDFs will be:

```
ml_age.hist(density=True, histtype='step', cumulative=False, linewidth=3.5, bins=10)
plt.xlabel('Age', fontsize=15)
```

```
plt.ylabel('CDF', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
fm_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5, bins=20)
plt.xlabel('Age', fontsize=15)
plt.ylabel('CDF', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
ml_age.hist(bins=10, density=True, histtype='stepfilled', alpha=.5) # default number of bins = 10
fm_age.hist(bins=10, density=True, histtype='stepfilled', alpha=.5, color=sns.desaturate("indianred", .75))
plt.xlabel('Age', fontsize=15)
plt.ylabel('Probability', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
ml_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5, bins=20)
fm_age.hist(density=True, histtype='step', cumulative=True, linewidth=3.5, bins=20, color=sns.desaturate("india
plt.xlabel('Age', fontsize=15)
plt.ylabel('CDF', fontsize=15)
plt.show()
```

[Show hidden output](#)

```
print("The mean sample difference is ", ml_age.mean() - fm_age.mean())
```

[Show hidden output](#)

✓ 2.4 Outliers

Outliers are data samples with a value that is far from the central tendency.

We can find outliers by:

- Computing samples that are *far* from the median.
- Computing samples whose value *exceeds the mean* by 2 or 3 standard deviations.

This expression will return a series of boolean values that you can then index the series by:

```
df['age'].median()
```

[Show hidden output](#)

Let's see how many outliers we can detect in our example:

```
len(df[(df.income == '>50K\n') & (df['age'] < df['age'].median() - 15)])
```

[Show hidden output](#)

```
len(df[(df.income == '>50K\n') & (df['age'] > df['age'].median() + 35)])
```

[Show hidden output](#)

If we think that outliers correspond to errors, an option is to trim the data by discarding the highest and lowest values.

```
df2 = df.drop(df.index[(df.income=='>50K\n') & (df['age']>df['age'].median() + 35) ])
df2.shape
```

[Show hidden output](#)

```
df2 = df.drop( df.index[(df.income=='>50K\n') &
((df['age'] > df['age'].median() + 35) | (df['age'] < df['age'].median() - 15))])
df2.shape
```

[Show hidden output](#)

```
ml1_age=ml50['age']
fm1_age=fm50['age']
```

```
ml2_age = ml1_age.drop(ml1_age.index[(ml1_age > df['age'].median() + 35) & (ml1_age > df['age'].median() - 15)])
fm2_age = fm1_age.drop(fm1_age.index[(fm1_age > df['age'].median() + 35) & (fm1_age > df['age'].median() - 15)])
```

```
mu2ml = ml2_age.mean()
std2ml = ml2_age.std()
md2ml = ml2_age.median()
# Computing the mean, std, median, min and max for the high-income male population
print("Men statistics: Mean:", mu2ml, "Std:", std2ml, "Median:", md2ml, "Min:", ml2_age.min(), "Max:", ml2_age.m
```

[Show hidden output](#)

```
mu3ml = fm2_age.mean()
std3ml = fm2_age.std()
md3ml = fm2_age.median()
# Computing the mean, std, median, min and max for the high-income female population
print("Women statistics: Mean:", mu2ml, "Std:", std2ml, "Median:", md2ml, "Min:", fm2_age.min(), "Max:", fm2_age
```

[Show hidden output](#)

```
print('The mean difference with outliers is: %4.2f.' % (ml_age.mean() - fm_age.mean()))
print("The mean difference without outliers is: %4.2f." % (ml2_age.mean() - fm2_age.mean()))
```

[Show hidden output](#)

Let's compare visually the age distributions before and after removing the outliers:

```
plt.figure(figsize=(13.4,5))

df.age[(df.income == '>50K\n')].plot(alpha=.25, color='blue')
df2.age[(df2.income == '>50K\n')].plot(alpha=.45,color='red')

plt.ylabel('Age')
plt.xlabel('Samples')
```

[Show hidden output](#)

Let's see what is happening near the mode:

```
import numpy as np

countx,divisionx = np.histogram(ml2_age, density=True)
county,divisiony = np.histogram(fm2_age, density=True)
```

```
import matplotlib.pyplot as plt

val = [(divisionx[i]+divisionx[i+1])/2 for i in range(len(divisionx)-1)]
plt.plot(val, countx-county, 'o-')
plt.title('Differences in promoting men vs. women')
plt.xlabel('Age',fontsize=15)
plt.ylabel('Differences',fontsize=15)
plt.show()
```

[Show hidden output](#)

One can see that the differences between male and female values are slightly negative before age 42 and positive after it. Hence, women tend to be promoted (receive more than 50K) earlier than men.

There is still some evidence for our hypothesis!

```
# The difference between the mean values of male and female populations.
print("Remember:\n We have the following mean values for men, women and the difference:\n0originally: ", ml_age.m
# The difference between the mean values of male and female populations.
print("For high-income: ", ml1_age.mean(), fm1_age.mean(), ml1_age.mean()- fm1_age.mean())
# The difference between the mean values of male and female populations.
print("After cleaning: ", ml2_age.mean(), fm2_age.mean(), ml2_age.mean()- fm2_age.mean())

print("\nThe same for the median:")
# The difference between the mean values of male and female populations.
print(ml_age.median(), fm_age.median(), ml_age.median()- fm_age.median())
# The difference between the mean values of male and female populations.
print(ml1_age.median(), fm1_age.median(), ml1_age.median()- fm1_age.median())
# The difference between the mean values of male and female populations.
print(ml2_age.median(), fm2_age.median(), ml2_age.median()- fm2_age.median())
```

Show hidden output